Optimizing KV Cache Management for Memory-Constrained LLM Inference: A Hybrid Eviction and Offloading Pipeline with Tensor-Parallel Scaling

Sagar Arun Kumar Dharma Swaroop Fareed Durgam

University of Pittsburgh
Operating Systems - CS2510, Spring 2026

Abstract

Large language model inference is fundamentally constrained by the memory footprint of the key-value (KV) cache, which grows linearly with sequence length and batch size. On memory-limited GPUs, this creates a tradeoff between serving capacity and throughput. Existing approaches treat eviction and offloading as alternatives—we argue they are complementary. We present a hybrid KV cache management pipeline that composes attention-based eviction, CPU offloading, and asynchronous prefetching with CUDA streams. We evaluate five system configurations on OPT-6.7B across single-GPU and tensor-parallel (TP=2) settings, and find that the hybrid approach not only eliminates out-of-memory failures but exhibits a scaling behavior where its relative throughput advantage grows with parallelism. We discuss the engineering challenges behind the implementation and outline a design for elastic tensor parallelism driven by KV cache memory pressure.

1 Introduction

The key-value cache is the dominant memory bottleneck in autoregressive large language model inference. During decoding, every previously generated token's key and value vectors must be stored so that the attention mechanism can attend to the full context. For OPT-6.7B, this amounts to roughly 512 KB per token across all 32 layers. Serving even a modest batch of 16 sequences at 512 generated tokens each requires over 4 GB of KV cache alone, on top of the 13 GB occupied by the model weights in half precision.

When the total memory demand exceeds the GPU's capacity, the system either refuses to serve the batch or crashes with an out-of-memory (OOM) error. This creates a fundamental tension in LLM serving: higher throughput requires larger batch sizes, but larger batches demand more KV cache memory. On commodity GPUs with 16–24 GB of VRAM, this tension becomes acute for models with 7B+ parameters.

Existing approaches address this pressure from two directions. Eviction-based methods such as H2O [1] keep only a fixed budget of cache entries on the GPU by discarding tokens with low cumulative attention scores. Offloading-based methods such as DejaVu [2] move the KV cache to CPU memory between decoding steps and stream it back when needed. Each approach has clear limitations: eviction sacrifices fidelity, while offloading introduces transfer overhead that can consume over 70% of wall-clock time.

In this work, we build a hybrid pipeline that composes these mechanisms rather than treating them as alternatives. We combine H2O eviction with DejaVu-style CPU offloading and add CUDA stream-based asynchronous prefetching with pinned memory to overlap data transfers with computation. We benchmark five configurations on OPT-6.7B, validate on Mistral-7B and Llama-3.1-8B, and extend to a manually implemented tensor-parallel (TP=2) setup. Our key finding is that eviction and offloading exhibit a synergy that amplifies with scale: on TP=2, the hybrid system outperforms the offload-only pipeline by 20%, reversing the single-GPU ordering.

2 Background and Related Work

2.1 KV Cache in Transformer Inference

In a standard transformer decoder, the attention operation at each layer requires the query vector of the current token to attend to the key and value vectors of all preceding tokens. Caching these vectors avoids redundant computation but creates a memory footprint that scales as $O(\text{batch} \times \text{layers} \times \text{heads} \times \text{seq_len} \times \text{head_dim})$. For OPT-6.7B (32 layers, 32 heads, head_dim=128, fp16), each token contributes 16 KB per layer or 512 KB across all layers. The KV cache size for a given configuration can be estimated as:

$$\text{KV}_{\text{size}} = L \times 2 \times B \times S \times H \times D \times \text{dtype} \quad (1)$$

where L is the number of layers, B is batch size, S is sequence length, H is the number of heads, D is head dimension, and dtype is 2 bytes for fp16.

2.2 H2O: Heavy-Hitter Oracle

H2O [1] observes that attention scores follow a power-law distribution: a small subset of tokens receive the majority of attention mass across heads and layers. It maintains a fixed

cache budget by evicting the token with the lowest cumulative attention score, protecting a recent window that may become important. In our implementation, we use a budget of 40% of the maximum sequence length and a recent window of 50% of the budget. H2O reduces GPU memory usage but introduces approximation error, and the eviction loop adds per-step compute overhead.

2.3 DejaVu: KV Cache Streaming

DejaVu [2] proposes offloading the KV cache to CPU memory between microbatches and streaming it back to the GPU when needed. The core idea is multi-microbatch pipeline parallelism: while one microbatch's forward pass executes on the GPU, the next microbatch's KV cache is being transferred from CPU. This overlap between computation and data movement is where the throughput benefit comes from. Without interleaving multiple microbatches, offloading strictly adds latency with no compensating gain.

2.4 Other Related Work

PagedAttention [3] reduces memory fragmentation through paged memory management and forms the basis of vLLM. InfiniGen [6] predicts which KV entries will be needed and prefetches selectively. SnapKV identifies important KV entries via attention pattern analysis. IMPRESS [4] explores importance-aware compression before offloading. LMCache provides a caching layer for KV reuse across requests. TurboQuant (Google, ICLR 2026) applies quantization to the KV cache for memory reduction. Our work differs in that we compose eviction and offloading as complementary tiers and study their interaction with tensor parallelism.

3 System Design

We evaluate five system configurations, each building on the previous. All systems use OPT-6.7B with eager attention (required to extract per-layer attention scores for H2O) and HuggingFace's DynamicCache API.

3.1 Baseline (GPU-only)

Standard autoregressive decoding with the full KV cache stored on the GPU. Each microbatch is processed sequentially: prefill computes the initial KV cache from the prompt, then decode generates tokens one at a time. This achieves the highest per-token throughput but OOMs when the cumulative KV cache across all microbatches exceeds available memory.

3.2 DejaVu (Vanilla CPU Offload)

After each microbatch's forward pass, the entire KV cache is moved to CPU memory via synchronous transfers. Before the next forward pass for the same microbatch, the cache is transferred back to GPU. This eliminates OOM but at a steep throughput cost: our measurements show 67–73% of inference time is spent on CPU↔GPU data transfers.

3.3 H2O (GPU-only Eviction)

Cumulative attention scores are tracked per token position across all layers and heads. After each decode step, if the cache exceeds the budget, the lowest-scoring token outside the recent window is evicted. Everything stays on GPU. H2O maintains near-baseline throughput ($\sim$31 tok/s) but still OOMs at large configurations because the full KV cache must reside on GPU during prefill.

3.4 DejaVu + Prefetch

This system introduces two CUDA streams alongside the default compute stream. The prefetch stream asynchronously copies the next microbatch's KV cache from CPU to GPU while the current microbatch's forward pass executes. The offload stream asynchronously copies the current microbatch's updated KV cache back to CPU after the forward pass completes. CPU-side buffers use pinned (page-locked) memory to enable non-blocking DMA transfers. The three-stage pipeline is:

$$\text{prefetch}(N{+}1) \rightarrow \text{compute}(N) \rightarrow \text{offload}(N)$$

3.5 Hybrid (H2O + DejaVu + Prefetch)

H2O eviction is applied after each decode step, before the cache is offloaded. By removing low-value tokens first, the cache that gets transferred to and from CPU is smaller, reducing both transfer time and CPU memory usage. The CUDA stream prefetch pipeline is retained. This system combines the memory efficiency of eviction with the OOM prevention of offloading, and the transfer volume reduction from eviction improves the effectiveness of the prefetch overlap.

4 Experimental Setup

4.1 Hardware and Software

Single-GPU. NVIDIA A100 with memory restricted to 18 GB via `torch.cuda.set_per_process_memory_fraction(0.45)`. PyTorch 2.x, HuggingFace Transformers, CUDA 12.x.

Multi-GPU. Two NVIDIA RTX A5000 GPUs (24 GB each) on RunPod. `torch.distributed` with NCCL backend. Transformers 4.50.3. Memory unrestricted (24 GB per GPU) for the TP=2 runs. Under a 16 GB restriction (simulating T4 GPUs), the baseline would OOM at larger configurations, demonstrating the need for offloading even with TP=2.

4.2 Models

Primary: OPT-6.7B (32 layers, 32 heads, head_dim=128, hidden=4096). ∼13 GB in fp16. This model was chosen because it is large enough to create genuine memory pressure on 16–18 GB GPUs while remaining practical for iterative experimentation.

Validation: Mistral-7B and Llama-3.1-8B are used to validate generalizability across model architectures (Section 5.4).

4.3 Workload

Prompts are drawn from WikiText-2 (test split), filtered for sequences longer than 200 tokens. We sweep microbatch counts $D \in \{8, 16, 32\}$ and generated token counts $N \in \{256, 512\}$.

4.4 Metrics

We report throughput (tok/s), peak VRAM (MB), CPU KV cache size (MB), and transfer overhead (%) where applicable. Throughput is computed as total generated tokens ($D \times N$) divided by wall-clock time from the start of the first decode step to the end of the last decode step.

5 Single-GPU Results

5.1 Throughput Comparison

The baseline achieves the highest throughput at approximately 37 tok/s but OOMs at D=16 N=512 and beyond. H2O maintains around 31 tok/s through eviction but also OOMs at large configurations because the full KV cache must reside on GPU during prefill. DejaVu eliminates OOM entirely by offloading to CPU, but throughput drops to 7.6–9.6 tok/s due to synchronous transfer overhead.

DejaVu+Prefetch achieves 16.1–18.7 tok/s, a consistent 1.9× improvement over vanilla DejaVu, confirming that CUDA stream prefetching with pinned memory cuts the offload bottleneck. The hybrid reaches 12.8–14.6 tok/s—slower than DejaVu+Prefetch due to the additional compute cost of attention score tracking and the per-token eviction loop, but it never OOMs and uses significantly less CPU memory.

5.2 Memory Efficiency

The offloading systems maintain peak VRAM between 13.4–14.1 GB regardless of batch size, while the baseline and H2O scale linearly until they OOM. The flat VRAM profile is a direct consequence of only holding one microbatch's KV cache on GPU at a time.

5.3 Transfer Overhead and CPU Memory

Vanilla DejaVu exhibits 67–73% transfer overhead, increasing with sequence length as the KV cache grows. The overhead is dominated by synchronous CPU↔GPU copies that block the compute stream.

The hybrid system achieves a 62% reduction in CPU memory usage compared with DejaVu and DejaVu+Prefetch. At D=32 N=512, the hybrid uses 4.7 GB of CPU KV cache versus 12.4 GB for the offload-only systems. This reduction is a direct consequence of H2O eviction shrinking the cache before offloading.

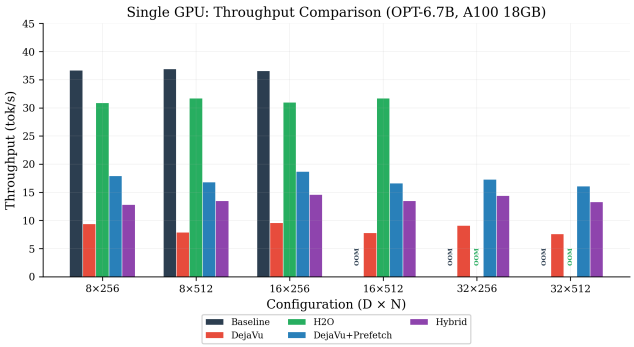


Figure 1: Single-GPU throughput. Baseline is fastest but OOMs early. Offloading systems trade throughput for capacity.

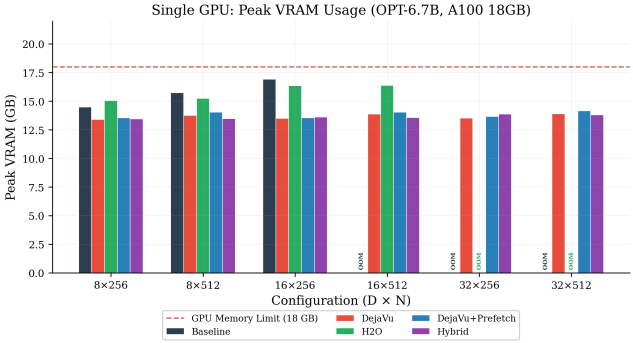


Figure 2: Peak VRAM usage. The red line marks the 18 GB GPU limit. Offloading systems remain flat.

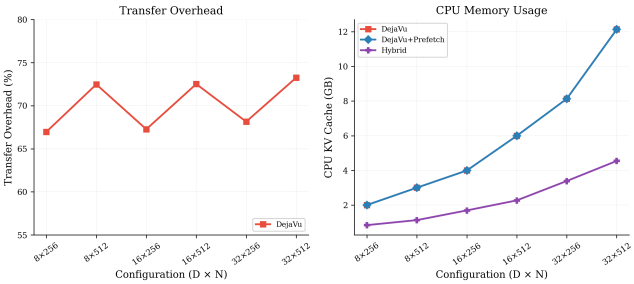


Figure 3: (Left) Transfer overhead for vanilla DejaVu. (Right) CPU KV cache comparison across offloading systems.

5.4 Generalizability Across Models

To confirm that the patterns observed with OPT-6.7B are not model-specific, we ran the same pipeline on Mistral-7B and

Llama-3.1-8B on the same A100 hardware. Both models show the same system ordering: baseline fastest but OOMs at scale, DejaVu trades throughput for capacity, prefetching recovers most of the offload cost, and the hybrid uses the least CPU memory. The relative throughput gaps between systems vary slightly across architectures, but the core findings hold.

6 Multi-GPU Evaluation (TP=2)

6.1 Tensor Parallelism Implementation

We implemented manual tensor parallelism across two NVIDIA RTX A5000 GPUs using `torch.distributed` with the NCCL backend. For each of the 32 attention layers, the Q, K, and V projection weight matrices are sliced along the output dimension (rows in `nn.Linear`'s transposed storage) so that each GPU computes attention over 16 of the 32 heads. The output projection is sliced along the input dimension (columns), and a `dist.all_reduce(SUM)` operation after each layer combines the partial results. The bias of the output projection is divided by the world size to ensure correct values after summation.

All other layers (FFN, layer norm, embeddings, lm_head) are replicated on both GPUs. We chose not to parallelize the FFN to simplify the implementation; this means memory savings come entirely from the KV cache halving, not from model weight distribution. The KV cache on each rank stores entries for 16 heads instead of 32, halving the per-GPU memory footprint. The prefetch and offload CUDA streams operate independently per rank with no cross-GPU KV communication.

We deliberately avoided frameworks such as Megatron-LM and DeepSpeed Inference because they own the forward pass and communication primitives, making it difficult to inject our custom KV pipeline. Additionally, Megatron-LM does not natively support OPT, and checkpoint conversion would have eaten up most of our limited development time. Our manual TP implementation required approximately 50 lines of new code (weight slicing + allreduce hooks) on top of the existing pipeline.

6.2 Throughput Results

The TP=2 baseline achieves approximately 50 tok/s with no OOM failures, a 35% improvement over the single-GPU baseline (∼37 tok/s). The baseline runs all configurations without offloading because the combined 48 GB of VRAM (24 GB × 2) provides sufficient headroom.

The most notable finding is the **throughput reversal** between the hybrid and offload-only systems. On a single GPU, the hybrid was approximately 24% slower than DejaVu+Prefetch (13.5 vs 17.9 tok/s at D=8 N=512). On TP=2, the hybrid is approximately 28% faster (23.1 vs 18.0 tok/s at the same configuration). This reversal is consistent across all configurations.

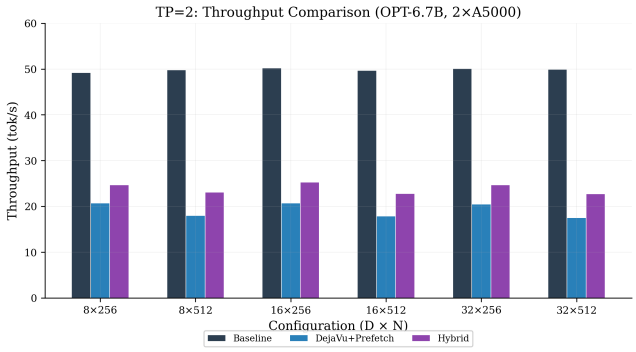


Figure 4: TP=2 throughput. The hybrid outperforms DejaVu+Prefetch, reversing the single-GPU ordering.

6.3 Analysis of the Throughput Reversal

The reversal has a clear mechanistic explanation. With TP=2, each rank runs H2O eviction over 16 heads instead of 32, halving the per-step eviction compute cost (fewer attention scores to track, smaller cumulative score tensors, fewer eviction candidates to search). Simultaneously, the post-eviction KV cache per rank is smaller in absolute terms, meaning the offload and prefetch transfers move less data per CUDA stream operation. The reduced transfer volume overlaps more effectively with the forward pass.

In short, tensor parallelism reduces the cost of eviction while amplifying its payoff (smaller transfers). This shifts the hybrid from being eviction-compute-bound (single GPU) to being transfer-efficient (TP=2). The DejaVu+Prefetch system, which does not evict, transfers the full per-rank KV cache and does not benefit from this cost reduction.

6.4 Scaling Comparison

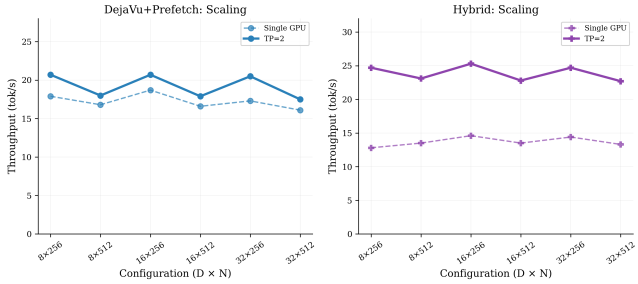


Figure 5: Scaling from single GPU to TP=2. Hybrid gains ∼71% throughput; Prefetch gains ∼15%.

Both systems benefit from tensor parallelism, but the hybrid benefits disproportionately. DejaVu+Prefetch improves from approximately 17 tok/s to 19 tok/s (∼15% gain), while the hybrid improves from roughly 14 tok/s to 24 tok/s (∼71% gain). This asymmetry suggests that the hybrid pipeline's single-GPU performance was more transfer-bound than compute-bound, and halving the per-rank KV size unlocked latent throughput that was previously hidden behind

transfer overhead.

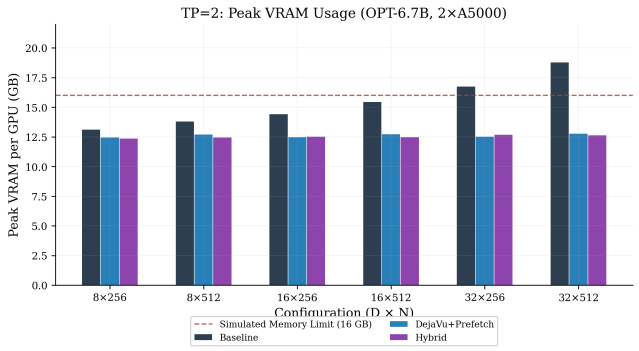


Figure 6: TP=2 peak VRAM per GPU. Offloading systems remain flat while the baseline scales with batch size.

7 Engineering Challenges and Lessons Learned

Building this pipeline involved a fair amount of engineering beyond algorithm design. A few challenges ate up development time but taught us useful lessons.

7.1 Pinned Memory and Page-Locking

CUDA's non-blocking memory transfers require the CPU-side memory to be pinned (page-locked). Without pinning, all transfers fall back to synchronous copies that block the compute stream. We initially observed no speedup from CUDA streams because the CPU buffers were not pinned. After adding `.pin_memory()` calls, throughput improved by 1.9× over vanilla DejaVu.

However, pinning itself has overhead. The `torch.Tensor.pin_memory()` call triggers a CUDA driver API call that page-locks the underlying memory pages. CUDA profiling confirmed that page-locking accounted for a noticeable chunk of remaining transfer time. A persistent pinned memory pool (allocate once, reuse across decode steps) would eliminate this overhead but was not implemented due to time constraints.

7.2 The torch.cat() Bottleneck

Appending new KV entries to the CPU-side cache using `torch.cat()` creates a new tensor on every decode step, copying all existing data plus the new entries. This is $O(N^2)$ in sequence length. At longer sequences, the concatenation overhead became visible in profiling. Experiments with chunked buffer transfers (writing new entries into a pre-allocated buffer via index assignment rather than concatenation) showed a ∼30% throughput improvement at chunk size 3. A transposed buffer layout `[num_layers, 2, batch, heads, max_seq, head_dim]` was identified as a further optimization (enabling contiguous memcpy per offload) but was not tested.

7.3 Distributed OOM Deadlocks

In the TP=2 setup, OOM errors during the forward pass created deadlocks. When one rank OOMed, it never reached the `all_reduce` call, while the other rank blocked indefinitely waiting for communication. Standard Python try/except blocks around the forward pass were insufficient because the deadlock occurs inside the NCCL collective, not in Python. We resolved this by splitting the benchmark into separate scripts per system, so a deadlock in one system did not prevent others from running.

8 Discussion

Our results show that composing eviction and offloading is not simply additive. The two mechanisms interact: eviction reduces the data volume that offloading must handle, and offloading provides the memory headroom that allows eviction to use a generous budget rather than an aggressive one. This synergy is amplified by tensor parallelism, where the per-rank reduction in both eviction cost and transfer volume produces a superlinear throughput gain for the hybrid system.

The throughput reversal between single-GPU and TP=2 is perhaps the most interesting finding. It suggests that the hybrid pipeline's architecture is inherently scalable: the eviction overhead that penalizes it on a single GPU diminishes with scale, while the transfer savings from eviction grow. If this trend continues to higher TP degrees, the hybrid system could become increasingly dominant.

A limitation of our evaluation is that we did not profile the TP=2 pipeline due to time constraints. While the throughput numbers are convincing, per-component profiling (eviction time, transfer time, allreduce time, compute time) would quantify exactly where the gains come from and whether the allreduce overhead is significant relative to the other costs.

Another limitation is that our TP implementation does not parallelize the FFN layers, meaning the model weights are not fully distributed. A complete TP implementation would further reduce per-GPU memory and improve scaling, but at the cost of an additional allreduce per layer.

9 Future Work

9.1 Dynamic Elastic Tensor Parallelism

We outline a direction for dynamic elastic tensor parallelism, where the system adjusts the TP degree at runtime based on KV cache memory pressure. Rather than statically provisioning GPUs, the system would start on a single GPU and scale to TP=2 or TP=3 only when the estimated KV footprint exceeds available memory, with the hybrid eviction+offload

pipeline as a final fallback tier. Existing systems such as DynamoLLM [5] perform dynamic TP switching but for energy optimization on high-end clusters, not memory-constrained inference. We leave the full design and evaluation of this approach to future work.

9.2 Offload Pipeline Optimizations

Several optimizations remain untested: (1) Pre-allocated fixed-size KV buffers to eliminate $O(N^2)$ `torch.cat` overhead; (2) Persistent pinned memory pools to remove page-locking latency; (3) True double buffering with two GPU-side buffers and two CUDA streams; (4) INT8/INT4 quantization of cold KV entries before offloading to reduce transfer size by 2–4$\times$. The chunked buffer approach already demonstrated a 30% gain; the transposed buffer layout could enable single contiguous memcpy per offload.

10 Conclusion

We presented a hybrid KV cache management pipeline for memory-constrained LLM inference that composes attention-based eviction (H2O), CPU offloading (DejaVu), and asynchronous prefetching with CUDA streams and pinned memory. On a single A100 GPU restricted to 18 GB, the hybrid system eliminates OOM failures across all tested configurations, reduces CPU memory usage by 62% compared with vanilla offloading, and the prefetch optimization achieves a 1.9$\times$ speedup over synchronous transfers.

On a manually implemented TP=2 setup with dual A5000 GPUs, the hybrid system achieves 22.7–25.3 tok/s, outperforming the offload-only pipeline (17.5–20.7 tok/s) by approximately 20%. This throughput reversal demonstrates that eviction and offloading exhibit a synergy that amplifies with scale: eviction becomes cheaper per rank while its transfer savings compound. These findings motivate further investigation into elastic tensor parallelism driven by KV cache pressure, where the serving system dynamically recruits GPUs based on workload demands rather than statically over-provisioning resources.

References

[1] Z. Zhang et al., "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," in *NeurIPS*, 2023.

[2] F. Strati et al., "DéjàVu: KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving," in *ICML*, 2024.

[3] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *SOSP*, 2023.

[4] "IMPRESS: Importance-based KV Cache Compression for LLM Inference," in *USENIX FAST*, 2025.

[5] J. Stojkovic et al., "DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency," in *HPCA*, 2025.

[6] G. Lee et al., "InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management," in *OSDI*, 2024.